Introduction to Low-Level Design (LLD)

Good code does more than produce the right output. Other developers should be able to understand it, change it, test it, and extend it without breaking unrelated functionality. This is why software design matters.

---

1. Why Software Design Matters

Consider an e-commerce application with a feature that allows customers to place orders. The initial flow seems straightforward:

1. Read the customer's cart.
2. Calculate the total price.
3. Check inventory.
4. Process payment.
5. Save the order.
6. Send a confirmation.

An initial implementation might put everything inside a single class:

```
public class OrderManager {

    public void placeOrder() {
        // Get items from cart
        // Calculate total amount
        // Check inventory
        // Process payment
        // Save order to database
        // Send confirmation email
    }
}
```

This may work for a small application. The difficulty appears when requirements change.

Suppose the product later needs discount coupons, payment through UPI or cards, cash on delivery, and multiple delivery partners. All of these features get added to `OrderManager`:

```
public class OrderManager {

    public void placeOrder() {
        // Validate cart
        // Calculate price
        // Apply coupons
        // Check stock
        // Select payment method
        // Process payment
        // Create order
        // Assign delivery partner
        // Save to database
        // Send notification
    }
}
```

The code may still run correctly, but it now handles too many separate responsibilities. This creates practical problems:

- **Changes become risky.** Modifying payment logic may accidentally affect the ordering workflow.
- **Testing becomes harder.** It is difficult to test stock checking independently when it is mixed with other operations.
- **Reusability suffers.** Another feature cannot easily reuse payment or notification logic.
- **Understanding the code takes longer.** A developer must follow a large method to find the behavior they need.
- **The class keeps growing.** Each new requirement adds more conditions and dependencies.

The issue is not whether the code works today. The issue is how difficult it becomes to change tomorrow. Requirements change throughout the life of a software product, so the structure of the code matters.

---

2. What Is Low-Level Design?

Low-Level Design (LLD) is the process of deciding how the internal components of a software system should be organized, what responsibilities each component should have, and how those components should interact to satisfy requirements.

LLD addresses questions such as:

- Which class should handle payments?
- Should inventory validation and payment processing live in the same class?
- What needs to change when a new payment method is introduced?
- Can order placement be tested without connecting to a real payment gateway?

Separating responsibilities

Instead of having one large `OrderManager`, the order flow can be divided into focused components:

Component	Responsibility
`OrderService`	Coordinates the steps involved in placing an order
`InventoryService`	Checks and manages inventory
`PaymentProcessor`	Provides a way to process payments
`OrderRepository`	Stores and retrieves order information
`NotificationService`	Sends order-related notifications

`OrderService` coordinates the workflow and delegates specialized work to the relevant components. This makes the code easier to understand, test, and modify.

This does **not** mean creating a separate class for every line of code. The goal is a sensible distribution of responsibilities, not the largest possible number of classes or design patterns.

---

3. Why Do We Need LLD?

Imagine two development teams working on the same product. One team focuses only on delivering features as quickly as possible. The other also spends time organizing responsibilities and keeping the code easy to change.

The first team may appear faster initially. Over time, however, repeated modifications to tightly connected code can slow it down. A well-structured design requires some thought upfront but can make later development more predictable.

LLD helps reduce four recurring problems:

1. **Difficult changes:** A small feature requires edits across several unrelated parts of the code.
2. **Unexpected regressions:** Modifying one behavior breaks another.
3. **Poor readability:** Developers struggle to identify where a particular responsibility belongs.
4. **Difficult testing:** Components cannot be verified independently without setting up the entire application.

The purpose of LLD is to make software **easier to understand, modify, extend, and test while preserving correct behavior**.

---

4. HLD vs. LLD vs. Machine Coding

Software design takes place at different levels. Consider building an entire e-commerce platform.

	High-Level Design (HLD)	Low-Level Design (LLD)	Machine Coding
Main focus	Overall system architecture	Internal structure and component interactions	Working implementation
Typical decisions	Major services, data storage, communication, scalability	Classes, responsibilities, methods, relationships, workflows	Writing code, choosing data structures, validating behavior, testing
Example	How the order system communicates with payment and inventory systems	How the order component delegates stock checks and selects a payment processor	Implementing the order classes and making the feature work

High-Level Design

HLD describes how the major parts of the system fit together. Typical questions include:

- Should the application use a modular monolith or multiple services?
- Where should information be stored?
- How should major components communicate?
- How should the architecture handle increasing traffic?

Low-Level Design

After the architecture is decided, the internal design still needs attention. For example:

- Which classes are needed inside order management?
- How should order states change?
- Which component validates a coupon?
- How is the correct payment processor selected?

Machine Coding

In an interview, **machine coding** usually means implementing a working solution within a limited period. The candidate is expected to translate a design into executable code, validate important conditions, and demonstrate that the core functionality works.

Two important distinctions:

- HLD is not limited to microservices. Monolithic applications also need architectural decisions.
- LLD is not simply the use of design patterns. Even a straightforward application benefits from careful responsibility assignment.

---

5. LLD vs. Data Structures and Algorithms (DSA)

DSA and LLD test related but different skills.

DSA problems often focus on finding a correct and efficient algorithm for a well-defined task. For example, selecting an appropriate data structure or minimizing time complexity.

LLD problems focus on organizing multiple pieces of behavior into a coherent system. The challenge includes identifying objects, assigning responsibilities, defining relationships, and preparing for reasonable requirement changes.

A developer may solve complex graph or dynamic programming problems and still struggle with a parking lot design if they place every operation in one giant class.

Another developer may identify clear responsibilities for spot allocation, parking fees, tickets, and availability, leading to a design that is easier to extend.

DSA still matters in LLD and machine coding. A good design must also use appropriate data structures and algorithms where needed. **LLD does not replace programming fundamentals; it adds design reasoning to them.**

---

6. Types of LLD-Related Interviews

A. Discussion-Oriented Design Interview

Example: Design a Parking Lot Management System.

The interviewer may expect the candidate to:

- Clarify the requirements and scope.
- Identify important domain objects.
- Explain responsibilities and relationships.
- Walk through the main workflows.
- Justify design choices when the requirements change.

Typical follow-up questions include:

- How is a parking spot allocated?
- Can a truck use a spot intended for a car?
- What happens when all suitable spots are occupied?
- How would a different pricing policy be introduced?

The main focus is design reasoning. Full working code may not be required; diagrams, pseudocode, or method signatures may be sufficient.

B. Machine Coding Interview

Example: Implement a simplified Splitwise application.

The requirements might include adding users, recording expenses, splitting amounts equally or by exact values, and displaying balances.

In this format, a candidate must **design and implement** the solution. Clear classes and interfaces matter, but the core functionality also needs to run correctly.

Common Evaluation Criteria

Area	What interviewers look for
Requirement understanding	Identifying scope and clarifying ambiguous behavior
Design quality	Sensible responsibilities and abstractions without unnecessary complexity
Correctness	Valid results and proper enforcement of business rules
Code quality	Readable, organized, understandable implementation
Extensibility	Ability to handle reasonable changes without rewriting unrelated code
Testing	Verification of normal, invalid, and edge cases
Communication and time management	Explaining trade-offs and delivering useful work within the time limit

The relative importance of these areas depends on the interview format.

Avoid Memorizing Entire Designs

Memorizing a fixed parking lot, elevator, or Splitwise solution is not enough. An interviewer can change a single requirement, making the memorized structure unsuitable.

A stronger approach is to understand **why** a class exists, **why** it owns a responsibility, and **what** would need to change if the requirements changed.

---

7. Requirement Gathering: Start with the Problem

Consider the interview prompt:

> Design a Parking Lot Management System.

It is tempting to immediately create `ParkingLot`, `Vehicle`, `ParkingSpot`, and `Ticket` classes. But choosing classes before clarifying scope means committing to a solution without fully understanding the problem.

Important questions include:

- Does the system manage one parking lot or many?
- Which vehicle types are supported: bikes, cars, trucks?
- Are there different parking spot sizes?
- Is advance reservation required?
- Does the system calculate parking fees? If yes, how?
- What should happen when the parking lot is full?

The answers influence which objects, operations, and rules are needed.

Functional Requirements

Functional requirements describe **what the system must do**.

Requirement	Expected behavior
Register parking spots	Configure available parking spots
Park a vehicle	Allocate an available, compatible spot
Issue a ticket	Associate a vehicle with its assigned spot
Exit a vehicle	End the parking session and free the spot
Check availability	Return the number of available spots
Handle full capacity	Reject a request when no suitable spot is free

Functional Requirements vs. Business Rules

A **functional requirement** describes an operation. A **business rule** defines the conditions under which that operation is valid.

Operation	Business rule
Park a vehicle	The spot must be compatible and unoccupied
Exit a vehicle	The vehicle must have an active parking session

The implementation must enforce these rules. A method that technically executes but violates a business rule is not correct.

---

8. Non-Functional Requirements and Invariants

Non-functional requirements describe **how well the system should operate** and what qualities or constraints matter.

Area	Parking lot consideration
Performance	How quickly should the system allocate a spot?
Concurrency	Can multiple entry gates allocate spots simultaneously?
Reliability	What happens if the application fails during entry?
Consistency	Can the same spot be assigned twice?
Scalability	How many vehicles and gates must the system handle?
Maintainability	How easily can a new vehicle type be supported?

Some properties, such as response time, can be expressed as measurable quality goals. Other conditions are better treated as **invariants**.

Invariant

An invariant is a condition that must remain true during all valid operations of the system.

For example:

> A parking spot must not be assigned to more than one active vehicle at the same time.

This rule should hold regardless of implementation details.

In a simple single-threaded program, checking availability before allocation may be enough. With concurrent requests, two threads could both see a free spot and

attempt to reserve it. The implementation would then need a mechanism to preserve the invariant under concurrent access.

9. Requirements, Constraints, and Assumptions

These terms have different meanings:

Term	Meaning	Parking lot example
Requirement	Something the solution must support	The system supports both bikes and cars
Constraint	A restriction on the implementation	Data must remain in memory; no database is allowed
Assumption	An explicit, reasonable choice when a detail is unspecified	Any available, compatible spot may be allocated

A good clarification question should reveal a decision that affects the design. Instead of asking a vague question such as “What are all the requirements?”, ask:

- Should we support multiple vehicle types or start with cars only?
- Can we allocate any compatible spot, or is a particular allocation strategy required?
- Do we need parking fee calculations, or only entry and exit?
- Should multiple gates be able to process requests concurrently?
- Is an in-memory implementation acceptable?

Specific questions make it easier to identify the actual scope and avoid spending time on unnecessary features.

Defining Version 1 Scope

In scope

- One parking lot
- Car and bike parking
- Compatible parking spots
- Spot allocation

- Ticket generation
- Vehicle exit
- Availability tracking

Out of scope

- Online reservations
- Dynamic pricing
- Payment processing
- Multiple parking lots
- Distributed coordination
- External database persistence

Explicit scope is important, especially in interviews with limited time. A good design solves the agreed problem rather than every possible future problem.

---

10. Design Thinking: From Requirements to Code

Once the requirements are clear, they can be converted into an object-oriented design.

Step 1: Write the Main Happy Flow

A **happy flow** describes the normal, successful path through a feature. For the parking lot:

1. A vehicle arrives.
2. The system looks for an available, compatible spot.
3. A spot is allocated.
4. A parking ticket is issued.
5. The parking session remains active until the vehicle exits.
6. At exit, the session is completed and the spot becomes available again.

This helps identify the operations the system must support before implementation begins.

Step 2: Discuss the Class Diagram

Identify the main domain objects, their responsibilities, and the relationships between them. Possible starting objects include `ParkingLot`, `ParkingSpot`, `Vehicle`, and `Ticket`.

The important questions are not merely which class names to use, but:

- Which object owns or manages the collection of parking spots?
- Which component finds and allocates a suitable spot?
- Where is the parking session or ticket information maintained?
- Which component updates availability when a vehicle exits?

A class diagram helps communicate the static structure. It is a design aid, not a substitute for understanding behavior.

Step 3: Use Sequence Diagrams When Useful

A sequence diagram explains the order in which objects interact for a particular scenario, such as parking a vehicle or handling an exit.

In an interview, it can be useful when the interviewer asks for a specific interaction or when the workflow is difficult to explain using only a class diagram. There is no need to draw every possible sequence diagram before coding.

Step 4: Start Coding Once the Core Design Is Clear

Do not spend the entire machine coding interview designing. A reasonable point to begin implementation is when you can explain:

- The main objects and what each one does
- How they interact in the primary workflow
- Which data structures are important
- The business rules and invariants that must be preserved

The design can be refined as needed. It does not have to predict every future requirement.

---

11. What Makes a Good LLD Solution?

A strong LLD solution combines several skills:

1. **Understanding requirements:** Clarify what needs to be built and what is outside the scope.
2. **Communicating the design:** Use class diagrams and, where useful, sequence diagrams to explain objects and interactions.
3. **Writing maintainable code:** Translate the design into readable implementation with appropriate responsibilities.
4. **Applying principles sensibly:** Use object-oriented principles, SOLID, and design patterns when they solve an actual design problem, not just to demonstrate that you know them.
5. **Working effectively with AI tools:** In AI-assisted machine coding rounds, AI may help generate boilerplate or implementation code. The developer remains responsible for requirements, design decisions, verification, and correctness.
6. **Explaining trade-offs:** Clearly communicate why one approach was chosen and how it would adapt to changing requirements.

A design is not good simply because it contains many classes, interfaces, or patterns. It is good when the responsibilities are clear, the requirements are satisfied, and the code remains manageable as the product evolves.

Quick Revision

- **HLD:** Decides the overall architecture and how major components connect.
- **LLD:** Decides the internal responsibilities, relationships, and interactions.
- **Machine coding:** Turns those decisions into a working implementation.
- **Functional requirements:** Define what the system must do.
- **Business rules:** Define when an operation is valid.
- **Non-functional requirements:** Describe quality goals and operating constraints.
- **Invariants:** Conditions that must remain true during valid operations.
- **Requirements gathering:** Comes before choosing classes and interfaces.

- **Good design:** Optimizes for clarity, correctness, testability, and reasonable change—not unnecessary complexity.